



Python Programming Fundamentals

Formatting Output

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. **f-Strings — Basics**
2. **f-String Format Specifiers**
3. **Alignment**
4. **Format Specifier Summary**
5. **Old-Style Formatting vs f-Strings**
6. **Pretty Printing Tables**
7. **Combining Input and Formatted Output**



Outline

- 1. f-Strings — Basics**
2. f-String Format Specifiers
3. Alignment
4. Format Specifier Summary
5. Old-Style Formatting vs f-Strings
6. Pretty Printing Tables
7. Combining Input and Formatted Output



1. f-Strings — Basics

f-strings (formatted string literals) are the modern way to embed values in strings.

```
name = "Alice"
```

```
age  = 25
```

```
gpa  = 3.87
```

```
# Basic embedding
```

```
print(f"Name: {name}")
```

```
print(f"Age: {age}")
```

```
print(f"GPA: {gpa}")
```

```
# Expressions in {}
```

```
print(f"In 5 years: {age + 5}")
```

```
print(f"GPA rounded: {round(gpa, 1)}")
```

```
print(f"Initials: {name[0]}.")
```



1. f-Strings — Basics

```
# Method calls in {}  
print(f"Upper: {name.upper()}")  
print(f"Words: {len(name.split())}")
```



Outline

1. f-Strings — Basics
- 2. f-String Format Specifiers**
3. Alignment
4. Format Specifier Summary
5. Old-Style Formatting vs f-Strings
6. Pretty Printing Tables
7. Combining Input and Formatted Output

2. f-String Format Specifiers

Add a `:` after the expression to specify formatting.

```
pi = 3.14159265
```

Decimal places

```
print(f"{pi:.2f}")      # 3.14
print(f"{pi:.4f}")      # 3.1416
print(f"{pi:.0f}")      # 3
```

Width

```
print(f"{pi:10.2f}")    # "      3.14" (width 10)
print(f"{42:05d}")      # "00042" (zero-padded)
```

Thousands separator

```
big = 1234567.89
print(f"{big:,.2f}")    # 1,234,567.89
```



2. f-String Format Specifiers

```
print(f"{big:,.0f}")      # 1,234,568
```

```
# Percentage
```

```
ratio = 0.8523
```

```
print(f"{ratio:.1%}")     # 85.2%
```

```
print(f"{ratio:.2%}")     # 85.23%
```




Outline

1. f-Strings — Basics
2. f-String Format Specifiers
- 3. Alignment**
4. Format Specifier Summary
5. Old-Style Formatting vs f-Strings
6. Pretty Printing Tables
7. Combining Input and Formatted Output

3. Alignment

```
name = "Alice"

# Left-align (default for strings)
print(f"|{name:<10}|")      # |Alice      |

# Right-align (default for numbers)
print(f"|{name:>10}|")      # |      Alice|

# Centre-align
print(f"|{name:^10}|")      # |  Alice  |

# Custom fill character
print(f"|{name:*<10}|")      # |Alice*****|
print(f"|{name:->10}|")      # |-----Alice|
print(f"|{name:-^10}|")      # |--Alice---|
```



3. Alignment



3. Alignment

```
name = "Alice"
```

```
# Left-align (default for strings)
```

```
print(f"|{name:<10}|")      # |Alice      |
```

```
# Right-align (default for numbers)
```

```
print(f"|{name:>10}|")      # |      Alice|
```

```
# Centre-align
```

```
print(f"|{name:^10}|")      # |  Alice  |
```

```
# Custom fill character
```

```
print(f"|{name:*<10}|")      # |Alice*****|
```

```
print(f"|{name:->10}|")      # |-----Alice|
```

```
print(f"|{name:-^10}|")      # |--Alice---|
```



3. Alignment

Practical — printing a table:

```
items = [("Apple", 0.99, 5), ("Bread", 2.49, 2), ("Milk", 1.29, 3)]
print(f"{'Item':<12} {'Price':>8} {'Qty':>5} {'Total':>8}")
print("-" * 36)
for name, price, qty in items:
    total = price * qty
    print(f"{name:<12} {price:>8.2f} {qty:>5} {total:>8.2f}")
```



Outline

1. f-Strings — Basics
2. f-String Format Specifiers
3. Alignment
- 4. Format Specifier Summary**
5. Old-Style Formatting vs f-Strings
6. Pretty Printing Tables
7. Combining Input and Formatted Output

4. Format Specifier Summary

Spec	Meaning	Example
:.2f	2 decimal places (float)	3.14
:d	Integer	42
:,	Thousands separator	1,234
:,.2f	Thousands + 2 decimals	1,234.57
:.1%	Percentage, 1 decimal	85.2%
:<10	Left-align, width 10	hello
:>10	Right-align, width 10	hello
:^10	Centre, width 10	hello
:05d	Zero-padded integer	00042
:e	Scientific notation	3.14e+00



Outline

1. f-Strings — Basics
2. f-String Format Specifiers
3. Alignment
4. Format Specifier Summary
- 5. Old-Style Formatting vs f-Strings**
6. Pretty Printing Tables
7. Combining Input and Formatted Output



5. Old-Style Formatting vs f-Strings

Python has several ways to format strings. Know them so you can read older code.

```
name = "Bob"  
score = 87.5
```

```
# Old %-style (Python 2 era – avoid in new code)
```

```
print("Name: %s, Score: %.1f" % (name, score))
```

```
# str.format() method (Python 3.0+)
```

```
print("Name: {}, Score: {:.1f}".format(name, score))
```

```
print("Name: {n}, Score: {:.1f}".format(n=name, s=score))
```

```
# f-string (Python 3.6+ – preferred)
```

```
print(f"Name: {name}, Score: {score:.1f}")
```

5. Old-Style Formatting vs f-Strings

Python has several ways to format strings. Know them so you can read older code.

```
name = "Bob"
score = 87.5

# Old %-style (Python 2 era – avoid in new code)
print("Name: %s, Score: %.1f" % (name, score))

# str.format() method (Python 3.0+)
print("Name: {}, Score: {:.1f}".format(name, score))
print("Name: {n}, Score: {s:.1f}".format(n=name, s=score))

# f-string (Python 3.6+ – preferred)
print(f"Name: {name}, Score: {score:.1f}")
```

Use f-strings for all new code — they are the most readable and fastest.



Outline

1. f-Strings — Basics
2. f-String Format Specifiers
3. Alignment
4. Format Specifier Summary
5. Old-Style Formatting vs f-Strings
- 6. Pretty Printing Tables**
7. Combining Input and Formatted Output



6. Pretty Printing Tables

Build neat tabular output using alignment specifiers.

```
def print_report(students):  
    """Print a formatted student report table."""  
    header = f"{'Name':<12} {'Score':>6} {'Grade':>6} {'Status':>8}"  
    separator = "-" * len(header)  
  
    print(header)  
    print(separator)  
  
    for name, score in students:  
        if score >= 90:    grade = "A"  
        elif score >= 80: grade = "B"  
        elif score >= 70: grade = "C"  
        elif score >= 60: grade = "D"  
        else:             grade = "F"
```



6. Pretty Printing Tables

```
status = "Pass" if score >= 60 else "Fail"  
print(f"{name:<12} {score:>6.1f} {grade:>6} {status:>8}")
```

```
students = [("Alice", 92.5), ("Bob", 74.0), ("Carol", 58.5), ("Dave", 85.0)]  
print_report(students)
```



Outline

1. f-Strings — Basics
2. f-String Format Specifiers
3. Alignment
4. Format Specifier Summary
5. Old-Style Formatting vs f-Strings
6. Pretty Printing Tables
- 7. Combining Input and Formatted Output**



7. Combining Input and Formatted Output

```
def get_float(prompt, minimum=None, maximum=None):
    while True:
        try:
            value = float(input(prompt))
            if minimum is not None and value < minimum:
                print(f"Minimum value is {minimum}")
                continue
            if maximum is not None and value > maximum:
                print(f"Maximum value is {maximum}")
                continue
            return value
        except ValueError:
            print("Please enter a number")

print("=== Loan Calculator ===")
principal = get_float("Loan amount (EUR): ", 0)
```



7. Combining Input and Formatted Output

```
rate          = get_float("Annual interest rate (%): ", 0, 100) / 100
years         = int(get_float("Term (years): ", 1, 30))

monthly_rate = rate / 12
months       = years * 12
monthly      = principal * monthly_rate / (1 - (1 + monthly_rate) ** -months)
total        = monthly * months

print(f"\nMonthly payment: €{monthly:,.2f}")
print(f"Total repaid:      €{total:,.2f}")
print(f"Interest paid:     €{total - principal:,.2f}")
```




Thanks for Watching - Any questions?

